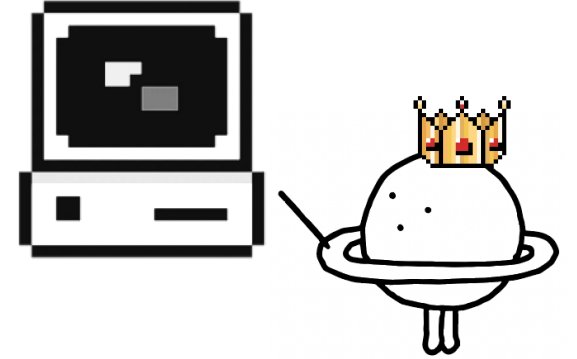


LOADING...



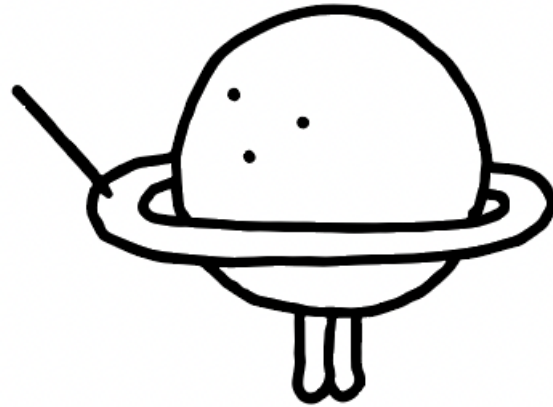
OS Synchronization

(feat. Java Synchronized)

우아한테크코스 BE 7기 메이



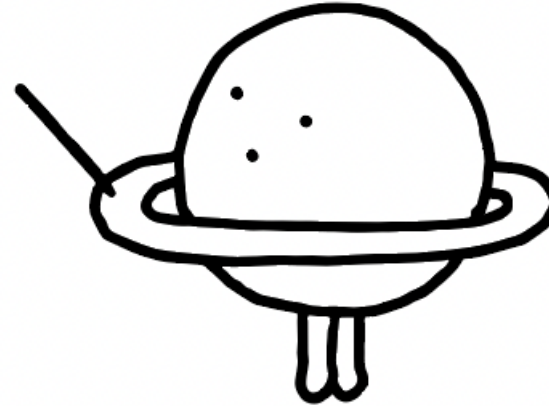
주제 선정 배경 & 주의점



목차

1. Race Condition, Critical Section
2. Atomic한 하드웨어 instruction
3. Mutex Lock & Semaphore
4. Monitor
5. Java Synchronized 구현

Race Condition, Critical Section



Race Condition

두 개 이상의 스레드가 공유 자원에 동시에 접근할 때, 실행 순서에 따라 결과가 달라지는 상황

잔액: 1000원

tA: 출금 500원

tB: 출금 800원

잔액 1000원 읽음

잔액 1000원 읽음

(1000 - 500)
뺄어씀

(1000 - 800)
뺄어씀

잔액: 500원

잔액: 200원

Race Condition

잔액: 1000원

tA: 출금 500원

tB: 출금 800원

잔액 1000원 읽음

잔액 1000원 읽음

$(1000 - 500)$
덧어씀

$(1000 - 800)$
덧어씀

잔액: 500원

잔액: 200원

Race Condition

잔액: 1000원

tA: 출금 500원

tB: 출금 800원

잔액 1000원 읽음

잔액 읽기 기다림..

$(1000 - 500)$
뺄어씀

tA가 사용하는동안
기다림

잔액: 500원

잔액 500원 읽음

...



Race Condition - OS

OS에서도 race condition 이 발생할 수 있는 상황이 다양하게 존재한다. 예를 들어...

pA: pid_t child = fork();

pB: pid_t child = fork();

request pid

next_available_pid = 25
(공유 변수)

child = 25

child = 25

Critical Section

```
fork() {
```

Atomic하게 연산되어야 하는 구간
(중간에 다른 스레드가 끼어들 수 없음)

```
...
```

```
기존 next_available_pid 얻어오기
```

```
next_available_pid++
```

```
pcb에 pid 할당
```

```
...
```

```
}
```

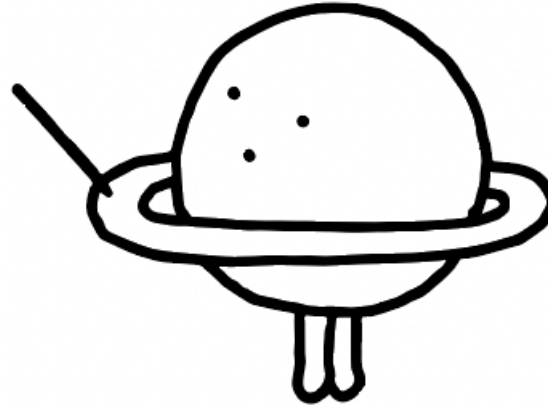
→ Critical Section



OS는 Critical Section 어케 보장함?

- **Atomic Instruction(TAS, CAS)**
 - **Mutex lock**
 - **Semaphore**
 - **Monitor**
- ...

Atomic한 하드웨어 instruction





잔액: 1000원

tA: 출금 500원

tB: 출금 800원

잔액 1000원 읽음

잔액 읽기 기다림..

잔액 사용 여부를 어떻게 확인?
(1000 - 500)
뺄어씀

tA가 사용하는동안
기다림

잔액: 500원

잔액 500원 읽음

...



잔액: 1000원

Lock: 0

tA: 출금 500원

tB: 출금 800원



Lock: 0 -> 1

잔액 1000원 읽음



(1000 - 500)
뺄어씀



Lock: 1 -> 0

잔액: 500원



Lock: 1

잔액 읽기 기다림..



tA가 사용하는동안
기다림

Lock: 0 -> 1

잔액 500원 읽음



...

Lock도 공유변수다

```
if (lock == 0) {           // ① 확인  
    lock = 1;              // ② 획득  
}
```



Critical Section

TAS(Test And Set), CAS(Compare And Swap)

CPU level에서 atomic하게 일어나는 연산

TAS(lock)

0(false) -> 1(true)로 set 시도
기존 값 return

```
bool TAS(bool *lock) {  
    bool old = *lock;    // 현재 값 읽기 (Test)  
    *lock = true;        // 무조건 true로 설정 (Set)  
    return old;          // 읽었던 값 반환  
}
```

CAS(lock, expected, new_value)

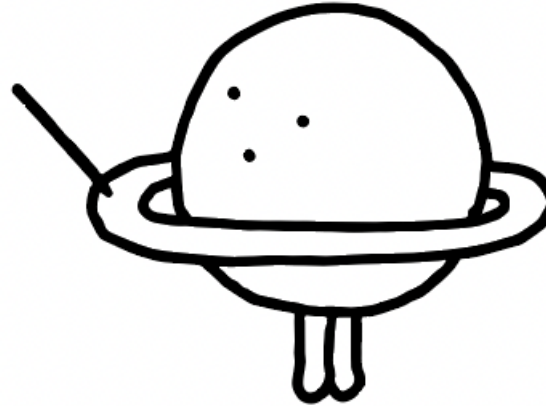
Lock의 값이 expected이면
new_value로 set
성공 여부 return

```
bool CAS(int *ptr, int expected, int new_value) {  
    if (*ptr == expected) {    // Compare  
        *ptr = new_value;      // Swap  
        return true;           // 성공  
    }  
    return false;              // 실패  
}
```

TAS(Test And Set), CAS(Compare And Swap)

**아무튼 TAS, CAS 이용하면
공유변수 확인 및 변경하는 과정이 원자적으로 처리된다!**

Mutex Lock & Semaphore





TAS로 critical section 보장하기

```
do {  
    while (test_and_set(&lock));  
    /* critical section */  
    lock = false;  
    /* remainder section */  
} while (true);
```

TAS로 critical section 보장하기 위해 필요한 코드

Mutex Lock

TAS/CAS을 이용해서, 하나의 스레드만 critical section에 진입을 보장하는 도구

복잡한 코드 캡슐화

```
mutex.lock()      // 획득 (acquire)
// Critical Section
mutex.unlock()    // 반환 (release)
```



lock(), unlock() 구현 예시

```
typedef struct {  
    int locked; // 0: 해제, 1: 점유  
} spinlock_t;  
  
void lock(spinlock_t *s) {  
    while (CAS(&s->locked, 0, 1) == false) {  
        // 실패하면 재시도 (busy wait)  
    }  
    // CAS 성공 → 락 획득  
}  
  
void unlock(spinlock_t *s) {  
    s->locked = 0;  
}
```

0이면 1로 set하고 true 반환
1이면 false 반환

Semaphore

Mutex lock(=> 0, 1만 가능)를 이용해 더욱 다양한 상황에서 사용할 수 있는 동기화 도구

Ex) 제한된 수의 커넥션(커넥션풀)에 접근하는 상황

```
semaphore_t sem;  
sem.count = 3; // 동시 접근 가능한 수
```

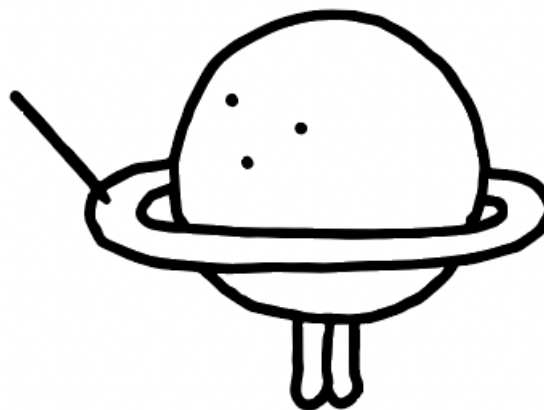
```
void use_connection() {  
    wait(&sem); // count-- , 0이면 대기  
  
    // 커넥션 사용  
  
    signal(&sem); // count++, 대기 중인 스레드 깨움  
}
```

count라는 공유변수 접근
→ Count에 대한 lock은
mutex lock으로 관리

```
typedef struct {  
    int count;  
    mutex_t mutex;  
} semaphore_t;
```



Monitor





Semaphore의 한계

```
semaphore_t sem;  
sem.count = 3; // 동시 접근 가능한 수  
  
void use_connection() {  
    wait(&sem);           // count-- , 0이면 대기  
  
    // 커넥션 사용  
  
    signal(&sem);         // count++, 대기 중인 스레드 깨움  
}
```

혹시라도 개발자가 wait()
/ signal() 빼먹을 수도
있지 않을까?



Monitor

동기화 로직을 개발자가 작성하지 않도록, 언어나 컴파일러 차원에서 보장해주는 도구

```
// count에 대한 monitor
```

```
monitor Counter {
```

공유 변수

```
    int count = 0;
```

```
    void increment() {
```

```
        count++;
```

```
    }
```

```
    void decrement() {
```

```
        count--;
```

```
    }
```

```
}
```

공유 변수에
접근하는 함수

```
// Counter 사용하는 쪽
```

```
void threadA() {
```

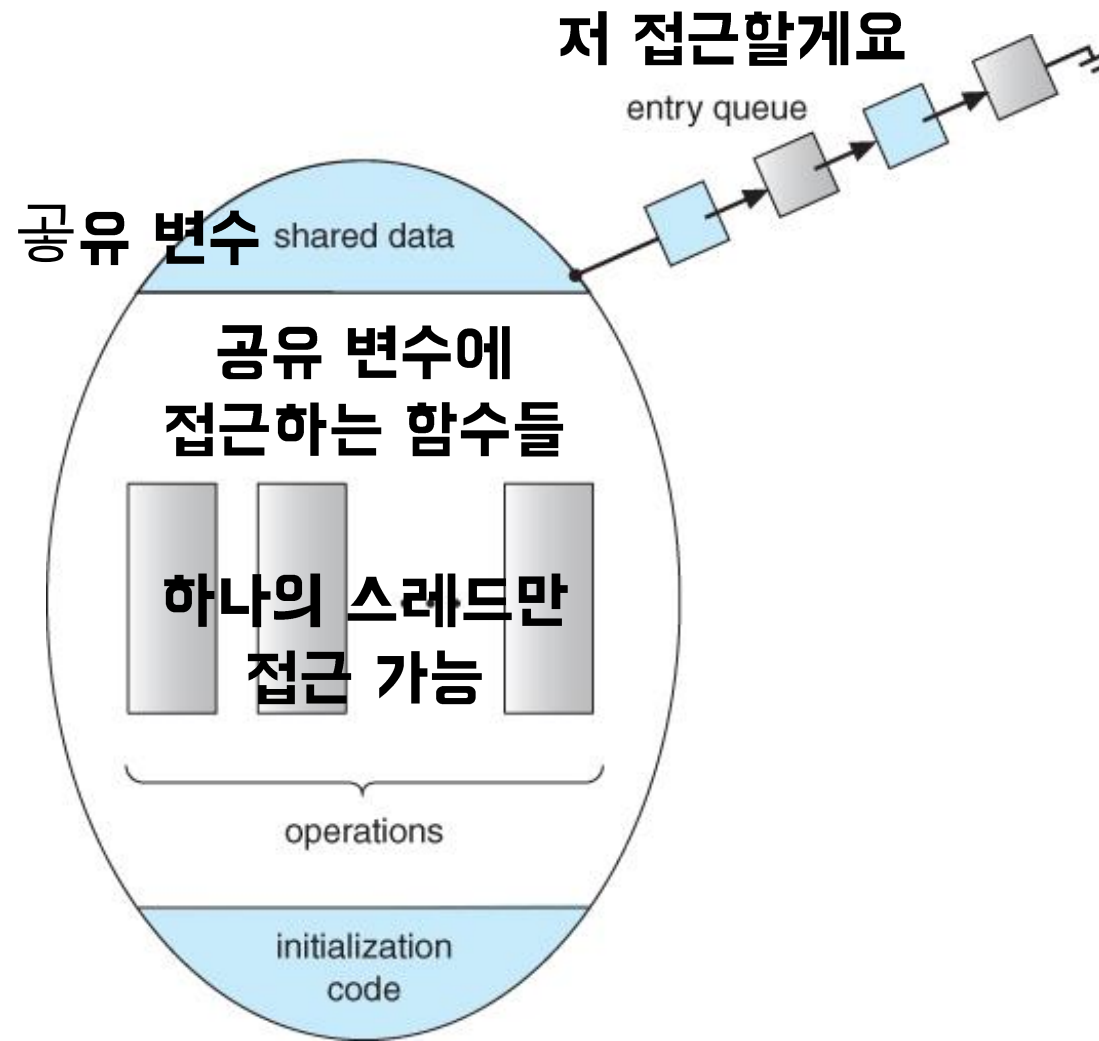
```
    // lock() 필요 없음
```

```
    Counter.increment(); // 락 신경 안 써도 됨
```

```
    // unlock() 필요 없음
```

```
}
```


Monitor 구조



```
// count에 대한 monitor
monitor Counter {
    공유 변수    int count = 0;

                void increment() {
                    count++;
    공유 변수에  }
    접근하는 함수
                void decrement() {
                    count--;
                }
            }
```

Count에 하나의 스레드만 접근한다는 것을 어떻게 보장할까?

```
monitor Counter {  
    int count = 0;  
  
    void increment() {  
        count++;  
    }  
  
    void decrement() {  
        count--;  
    }  
}
```

컴파일러가 count에 대한
락 관리하는 코드 추가



```
class Counter {  
    int count = 0;  
    mutex_t mutex;  
  
    void increment() {  
        mutex_lock(&mutex);  
        count++;  
        mutex_unlock(&mutex);  
    }  
  
    void decrement() {  
        mutex_lock(&mutex);  
        count--;  
        mutex_unlock(&mutex);  
    }  
}
```



OS Synchronization Tools 정리

추상화 레벨 ↑
개발자 편의성 ↑

TAS/CAS

- 하드웨어가 지원해주는 원자적 명령어
- lock = 0 / 1 설정을 원자적으로 처리 가능



0.1초 만에 포스트잇을 확인하고 바꿔 치기하는 초능력

Mutex

- TAS/CAS로 lock을 획득/해제 하는 과정을 추상화
- 값은 0, 1만 가능



단 하나뿐인 화장실 열쇠를 가진 사람만 이용

Semaphore

- Integer 값을 가질 수 있는 공유 변수 관리 도구
- mutex 사용해서 공유 변수 접근 관리



주차장 빈자리 개수를 보고 들어가는 전광판

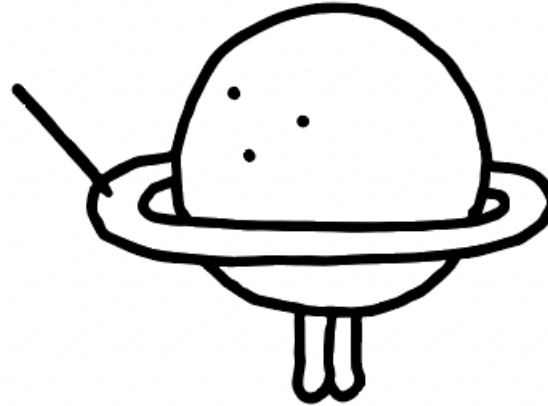
Monitor

- 컴파일러가 동기화 로직 추가해줌
- 모니터 내에 공유변수 있고 모니터 내 함수로만 접근 가능



입구에서 한 명씩 들여보내 주는 클럽 보안 요원

Java Synchronized 구현



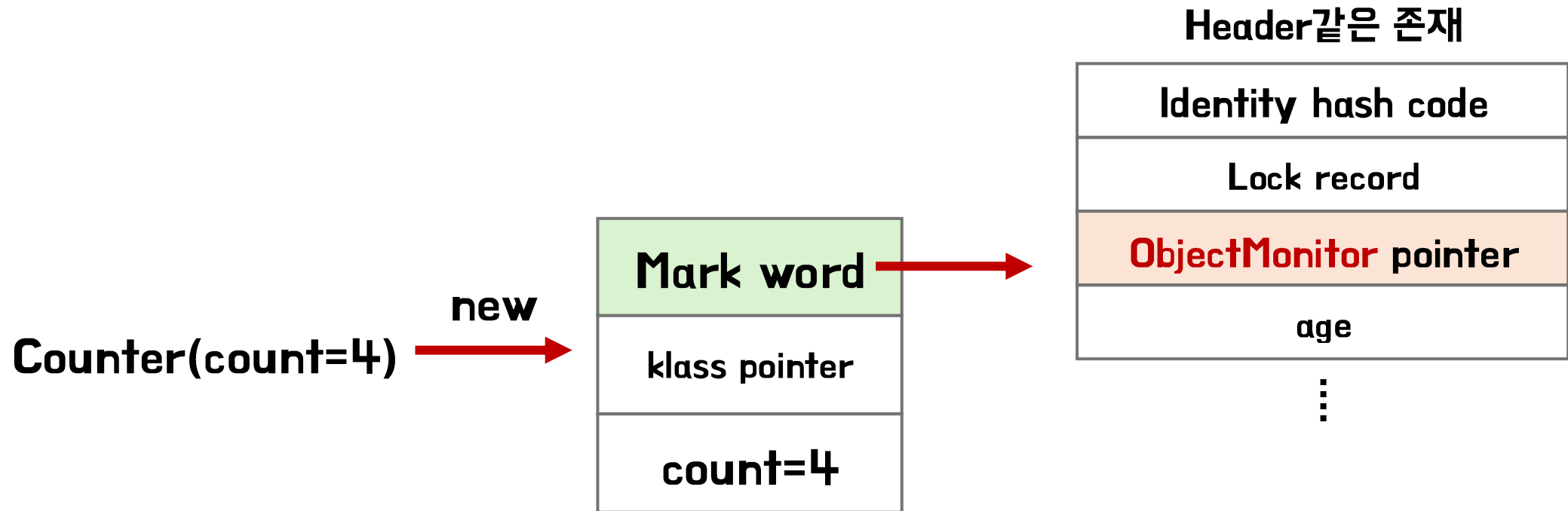
Synchronized

```
public class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int getCount() {  
        return count;  
    }  
}
```

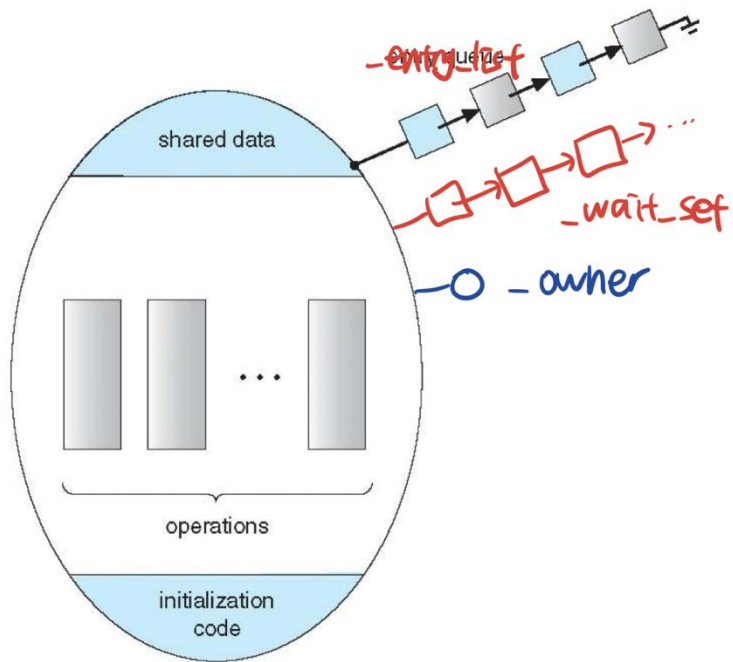
여러 스레드가 동시에 increment() 호출할 때,
어떻게 counter에 대한
단일 스레드 접근이 보장되는걸까?

Counter 객체에 하나의 스레드만 접근 가능

Mark word 구조



ObjectMonitor 코드



모니터를 점유중인 스레드

```
int64_t volatile _owner; // Either owner_id of owner, NO_OWNER, ANONYMOUS_OWNER or DEFLATER_MARKER.
volatile uint64_t _previous_owner_tid; // thread id of the previous owner of the monitor
// Separate _owner and _next_om on different cache lines since
// both can have busy multi-threaded access. _previous_owner_tid is only
// changed by ObjectMonitor::exit() so it is a good choice to share the
// cache line with _owner.
#define PAD_MINUS_SIZE(1, OM_CACHE_LINE_SIZE, sizeof(void* volatile) +
                        sizeof(volatile uint64_t));
```

```
ObjectMonitor* _next_om; // Next ObjectMonitor* linkage
volatile intx _recursions; // recursion count, 0 for first entry
ObjectWaiter* volatile _entry_list; // Threads blocked on entry or reentry.
// The list is actually composed of wait-nodes,
// acting as proxies for Threads.
ObjectWaiter* volatile _entry_list_tail; // _entry_list is the head, this is the tail.
int64_t volatile _succ; // Heir presumptive thread - used for futile wakeup throttling
```

```
volatile int _SpinDuration;
```

```
int _contentions; // Number of active contentions in enter(). It is used by is_busy()
// along with other fields to determine if an ObjectMonitor can be
// deflated. It is also used by the async deflation protocol. See
// ObjectMonitor::deflate_monitor().
int64_t _unmounted_vthreads; // Number of nodes in the _entry_list associated with unmounted vthreads.
// It might be temporarily more than the actual number but never less.
```

```
OopHandle _object_strong; // Used to protect object during preemption on class initialization
```

```
ObjectWaiter* volatile _wait_set; // LL of threads waiting on the monitor - wait()
volatile int _waiters; // number of waiting threads
volatile int _wait_set_lock; // protects wait set queue - simple spinlock
volatile int _object_strong_lock; // protects setting of _object_strong
```

모니터를 점유하려고 대기중인 스레드들

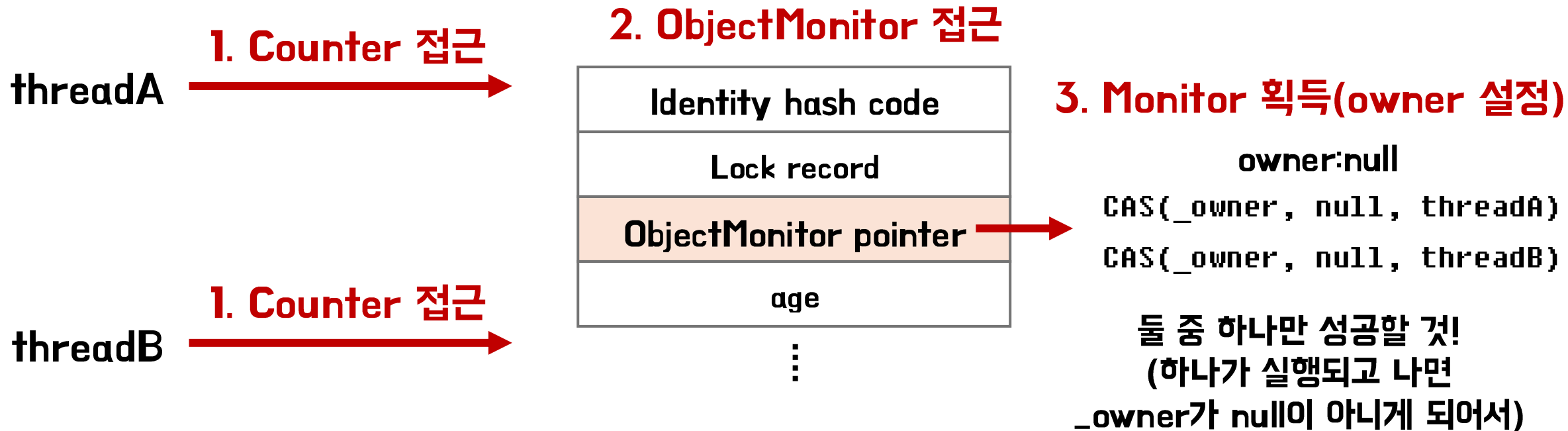
모니터를 점유했었는데 wait()을 호출해서 기다리고 있는 스레드들



Java Monitor lock 획득 과정

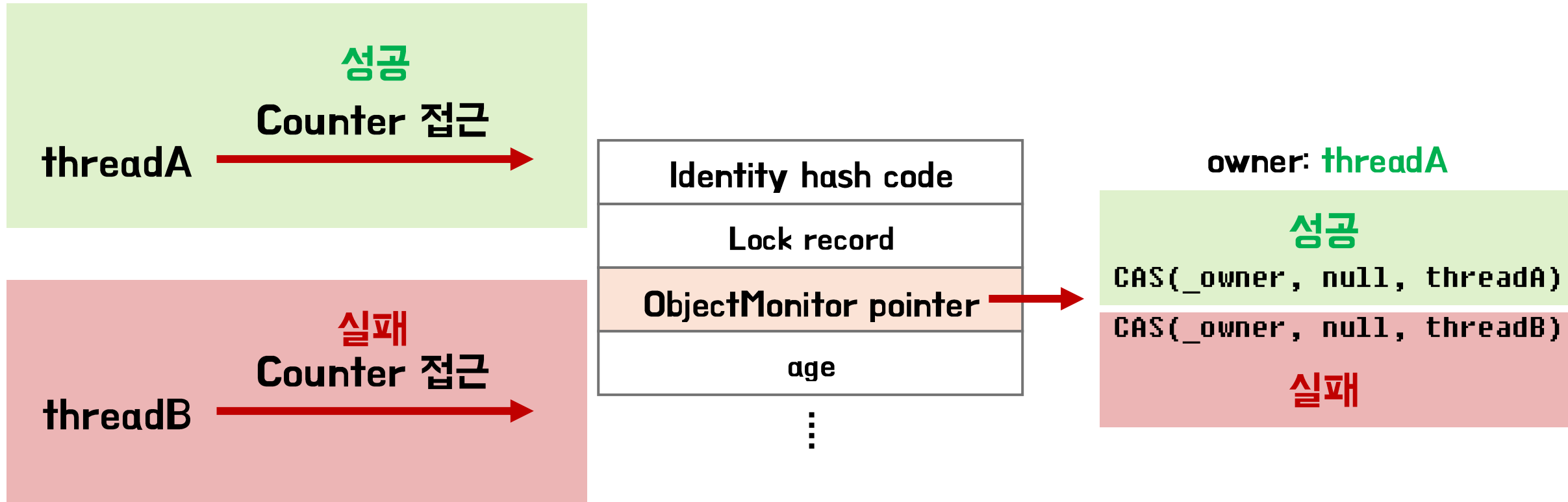
```
public synchronized void increment() {  
    count++;  
}
```

threadA, threadB
동시 접근하는 상황 가정





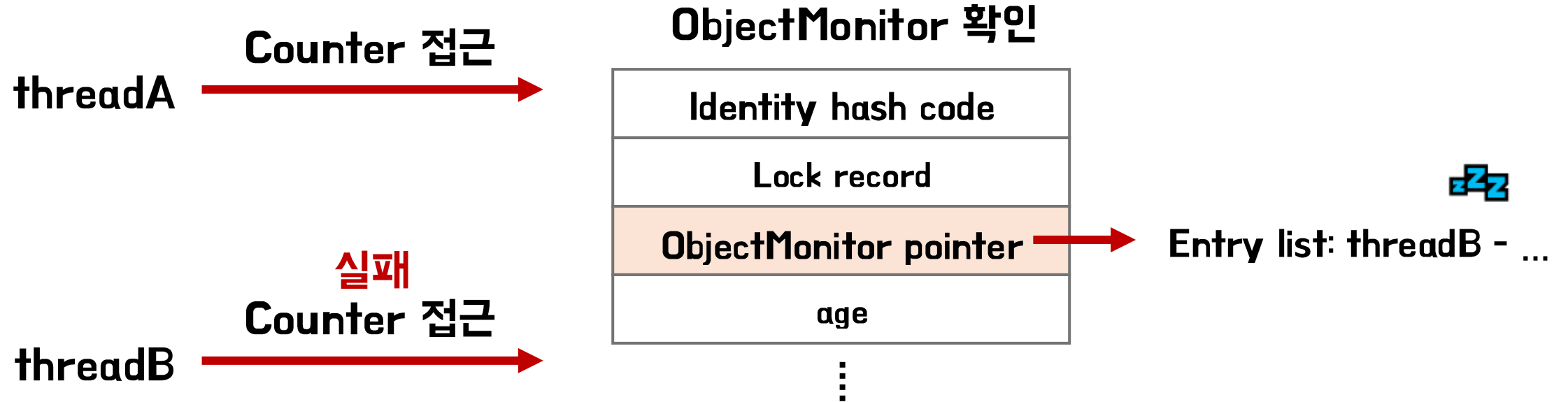
Java Monitor lock 획득 과정





Java Monitor lock 획득 과정

```
public synchronized void increment() {  
    count++; ..... 사용중..  
}
```





Java Monitor lock 획득 과정

```
public synchronized void increment() {  
    count++;  
} ..... 사용 끝!
```

threadA

threadB → Counter 접근

ObjectMonitor 확인

Identity hash code
Lock record
ObjectMonitor pointer
age
⋮

owner: null

unpark()

Entry list: threadB - ...



Java Monitor lock 획득 과정

threadA

threadB

Counter 접근

ObjectMonitor 확인

Identity hash code
Lock record
ObjectMonitor pointer
age

⋮

owner: null

CAS(_owner, null, threadB)

성공



Object.java의 Monitor 관련 메서드

```
public final void wait() throws InterruptedException {
```

```
public final native void notify();
```

```
public final native void notifyAll();
```

```
class BoundedBuffer {
```

```
    private final Queue<Integer> buffer = new LinkedList<>();
```

```
    private final int CAPACITY = 5;
```

```
    public synchronized void produce(int item) throws InterruptedException {
```

```
        while (buffer.size() == CAPACITY) {
```

```
            wait(); // 버퍼 꽉 차면 대기 → wait_set에 추가되고 잠듦
```

```
        }
```

```
        buffer.add(item);
```

```
        System.out.println("produced: " + item);
```

```
        notifyAll(); // 소비자 깨움 → wait_set의 스레드들 entry_list로 옮기고  
                                     나는 계속 실행
```

```
    }
```

```
    ...
```

```
}
```

-> Signal and continue

Q&A

